Natural Computing

LMU Munich
summer term 2025

Thomas Gabor

Can cellular automata
explain the universe?

Consider the following variation of a 1D cellular automaton (cf. Definitions 1 and 2 on the definition sheet): Let the state space $\mathcal{X} = (V \to \mathbb{Z} \times \mathbb{Z})$. Let

$$f\big((v_l, p_l), (v_c, p_c), (v_r, p_r)\big) = \big(v_l^+ + v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor - \lfloor \frac{(p_r - p_c)^+}{2} \rfloor,$$

$$p_c - |v_c| + v_l^+ - v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor + \lfloor \frac{(p_r - p_c)^+}{2} \rfloor - \lfloor \frac{(p_c - p_l)^+}{2} \rfloor - \lfloor \frac{(p_c - p_r)^+}{2} \rfloor\big)$$

where $x^+ = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases}$ and $x^- = \begin{cases} x & \text{if } x < 0, \\ 0 & \text{otherwise,} \end{cases}$ for any $x \in \mathbb{Z}$.

Note: $\lfloor x \rfloor$ for $x \in \mathbb{R}$ is x rounded down to an integer as implemented in a typical `floor` function (see Python, e.g.).
From the start configuration of this 1d cellular automaton with 5 cells below, compute the next 3 time steps.

$x_0 =$

$(0,0)$	$(+1,+1)$	$(0,0)$	$(-1,+1)$	$(0,0)$

Example 1 (Cellular Automaton for 1D Particle Collision, inspired by [1]).
Consider the following variation of a 1D cellular automaton (cf. Definitions 1 and 2 on the definition sheet): Let the state space $\mathcal{X} = (V \to \mathbb{Z} \times \mathbb{Z})$. Let

$$f\big((v_l, p_l), (v_c, p_c), (v_r, p_r)\big) = (v_l^+ + v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor - \lfloor \frac{(p_r - p_c)^+}{2} \rfloor,$$
$$p_c - |v_c| + v_l^+ - v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor + \lfloor \frac{(p_r - p_c)^+}{2} \rfloor - \lfloor \frac{(p_c - p_l)^+}{2} \rfloor - \lfloor \frac{(p_c - p_r)^+}{2} \rfloor)$$

where $x^+ = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases}$ and $x^- = \begin{cases} x & \text{if } x < 0, \\ 0 & \text{otherwise,} \end{cases}$ for any $x \in \mathbb{Z}$.

Note: $\lfloor x \rfloor$ for $x \in \mathbb{R}$ is x rounded down to an integer as implemented in a typical `floor` function (see Python, e.g.).
From the start configuration of this 1d cellular automaton with 5 cells below, compute the next 3 time steps.

$(0,0)$	$(+1,+1)$	$(0,0)$	$(-1,+1)$	$(0,0)$

Example 1 (Cellular Automaton for 1D Particle Collision, inspired by [1]). Consider the following variation of a 1D cellular automaton (cf. Definitions 1 and 2 on the definition sheet): Let the state space $\mathcal{X} = (V \to \mathbb{Z} \times \mathbb{Z})$. Let

$$f\big((v_l, p_l), (v_c, p_c), (v_r, p_r)\big) = (v_l^+ + v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor - \lfloor \frac{(p_r - p_c)^+}{2} \rfloor,$$
$$p_c - |v_c| + v_l^+ - v_r^- + \lfloor \frac{(p_l - p_c)^+}{2} \rfloor + \lfloor \frac{(p_r - p_c)^+}{2} \rfloor - \lfloor \frac{(p_c - p_l)^+}{2} \rfloor - \lfloor \frac{(p_c - p_r)^+}{2} \rfloor)$$

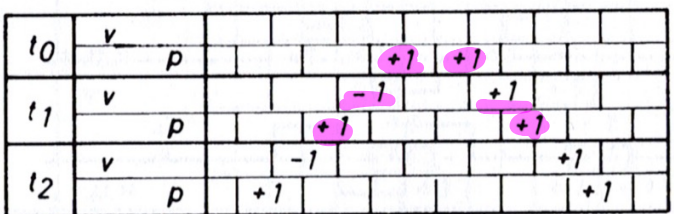
where $x^+ = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases}$ and $x^- = \begin{cases} x & \text{if } x < 0, \\ 0 & \text{otherwise,} \end{cases}$ for any $x \in \mathbb{Z}$.

Note: $\lfloor x \rfloor$ for $x \in \mathbb{R}$ is x rounded down to an integer as implemented in a typical `floor` function (see Python, e.g.).
From the start configuration of this 1d cellular automaton with 5 cells below, compute the next 3 time steps.

$(0,0)$	$(+1,+1)$	$(0,0)$	$(-1,+1)$	$(0,0)$
$(0,0)$	$(0,0)$	$(0,2)$	$(0,0)$	$(0,0)$
$(0,0)$	$(-1,+1)$	$(0,0)$	$(+1,+1)$	$(0,0)$
$(-1,+1)$	$(0,0)$	$(0,0)$	$(0,0)$	$(+1,+1)$

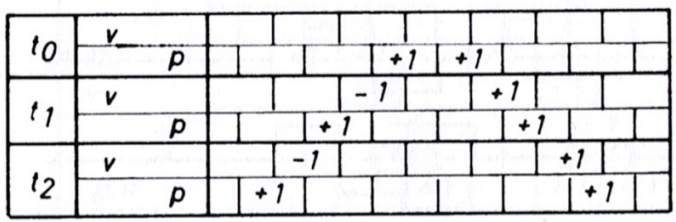
```python
from math import floor

def eplus(x):
    return x if x > 0 else 0

def eminus(x):
    return x if x < 0 else 0

def f(left,center,right):
    vl,pl = left
    vc,pc = center
    vr,pr = right
    pressure_from_left = floor(eplus(pl-pc)/2)
    pressure_from_right = floor(eplus(pr-pc)/2)
    pressure_towards_left = floor(eplus(pc-pl)/2)
    pressure_towards_right = floor(eplus(pc-pr)/2)
    vnew = eplus(vl) + eminus(vr) + pressure_from_left - pressure_from_right
    pnew = pc - abs(vc) + eplus(vl) - eminus(vr) + pressure_from_left + pressure_from_right - pressure_towards_left - pressure_towards_right
    return vnew, pnew

def apply(vector, f):
    new_vector = []
    for e,element in enumerate(vector):
        new_element = f(vector[e-1], element, vector[(e+1)%len(vector)])
        new_vector.append(new_element)
    return new_vector

rounds = range(4)
vector = [(0,0), (+1,+1), (0,0), (-1,+1), (0,0)]
for _ in rounds:
    print(vector)
    vector = apply(vector,f)
```

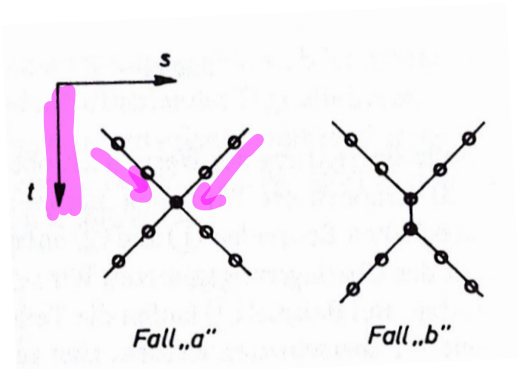
state space $\mathcal{X} = (V \to \mathbb{Z} \times \mathbb{Z})$



Zuse. Rechnender Raum. 1969

state space $\mathcal{X} = (V \to \mathbb{Z} \times \mathbb{Z})$





Zuse. Rechnender Raum. 1969

Konrad Zuse

1910 – 1995

- built first programmable computer (Zuse Z3)
- designed first high-level programming language (Plankalkül)

source: Wikipedia

What about non-local information?

What should that even be?

real information can only spread at light speed (max)

Quantum Computing

What is quantum computing?

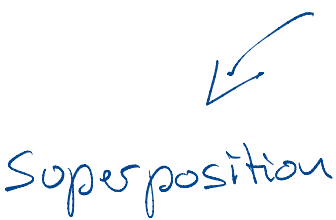
using quantum effects to do computing

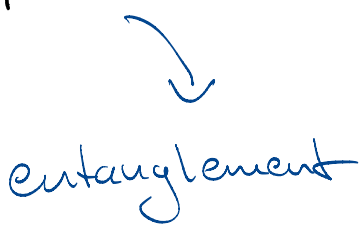
What is quantum computing within natural computing?

interesting physical process $\Longrightarrow$ build a computer

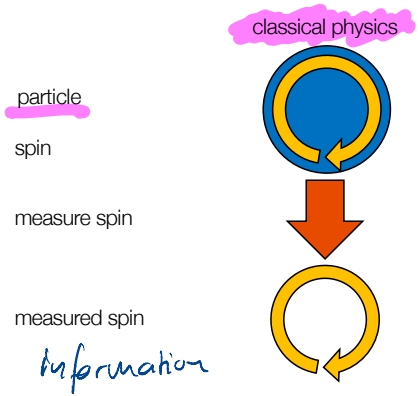
- mechanical
- electric
- quantum

two strange
quantum effects

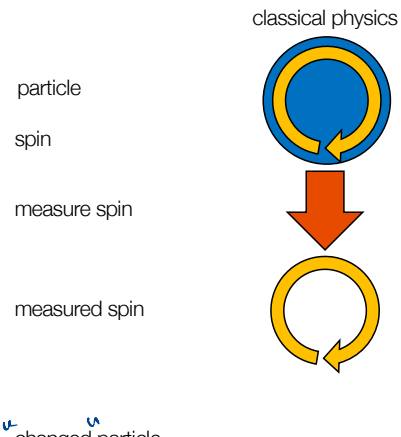


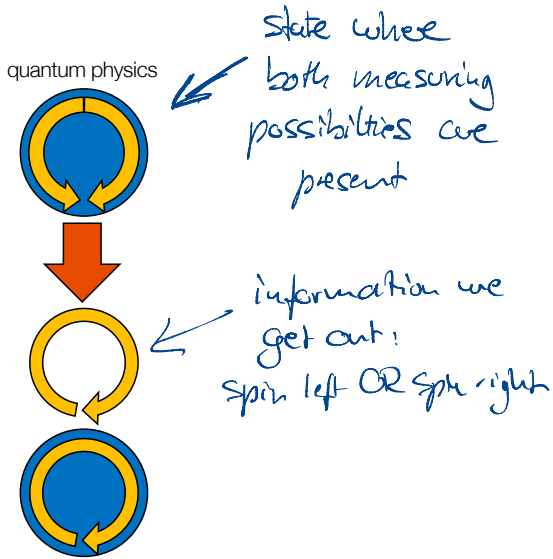


Superposition



Superposition





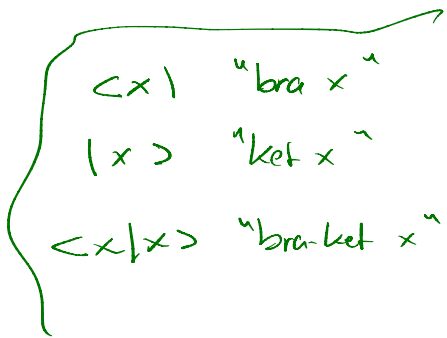
Definition 5 (quantum state)**.** Let $\mathcal{X}$ be an arbitrary state space $\mathcal{X}$. Let $\mathcal{Q}(\mathcal{X})$ be the space of superposition states with bases in $\mathcal{X}$, i.e., each $q \in \mathcal{Q}(\mathcal{X})$ has the form

$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

where $c_x \in \mathbb{C}$ for any $x \in \mathcal{X}$ is called an amplitude so that

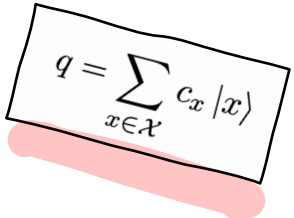
$$\sum_{x \in \mathcal{X}} |c_x|^2 = 1$$

and $|x\rangle$ (read "ket x") denotes a vector corresponding to the state $x \in \mathcal{X}$.



We use the following terminology:

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.



We use the following terminology:

$$q = \sum_{x \in \mathcal{X}} c_x \, |x\rangle$$

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.
- If $\mathcal{X} = \mathcal{X}_A \times \mathcal{X}_B$, i.e., $\mathcal{X}$ can be constructed as a product of two other state spaces $\mathcal{X}_A, \mathcal{X}_B$, then it follows that q has the form

$$q = \sum_{\substack{x_A \in \mathcal{X}_A, \\ x_B \in \mathcal{X}_B}} c_{x_A, x_B} \, |x_A\rangle \otimes |x_B\rangle$$

where $\otimes$ is the tensor product. If q cannot be written as

$$q = \Big(\sum_{x_A \in \mathcal{X}_A} c_{x_A} \, |x_A\rangle \Big) \otimes \Big(\sum_{x_B \in \mathcal{X}_B} c_{x_B} \, |x_B\rangle \Big),$$

then we call q an *entangled* state.

- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as

$$q = c_0 \left|0\right\rangle + c_1 \left|1\right\rangle$$

for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.

$$q = \sum_{x \in \mathcal{X}} c_x \left|x\right\rangle$$

- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as

$$q = c_0 \left|0\right\rangle + c_1 \left|1\right\rangle$$

for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.

- If $\mathcal{X} = \{0, 1\}^n$ for any $n \in \mathbb{N}$ (i.e., any $x \in \mathcal{X}$ is a bit register), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit register* of size n and can be written as

$$q = \sum_{i=0}^{2^n - 1} c_i \left|i\right\rangle = c_{0\ldots0} \left|0 \ldots 0\right\rangle + \cdots + c_{1\ldots1} \left|1 \ldots 1\right\rangle$$

where we usually denote i in binary as illustrated above.

$$q = \sum_{x \in \mathcal{X}} c_x \left|x\right\rangle$$

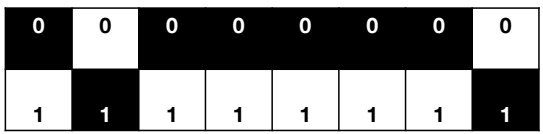
- When we *measure* a quantum state $q \in \mathcal{Q}(\mathcal{X})$, we generate classical state $M(q) \in \mathcal{X}$ so that for all $x \in \mathcal{X}$ it holds that

$$\Pr(M(q) = x) = |c_x|^2.$$

$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

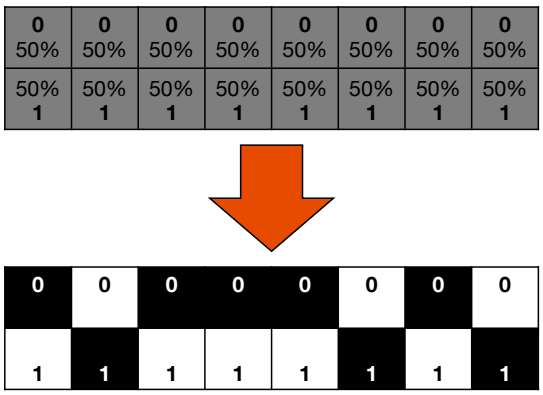
"equal qubit" $q = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$

$$\Pr(M(q) = 1) = \left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}$$



ASCII letter A = 01000001

qubit register



all ASCII chars in superposition with equal amplitude

measuring

ASCII char "E" with probability 1/256